Using snowflakeR in Workspace Notebooks

Snowflake Workspace Notebooks provide a managed Jupyter environment with a Python kernel. `snowflakeR` works seamlessly in this environment via `rpy2` and `%%R` magic cells, giving R users full access to Snowflake ML features without leaving the notebook.

This vignette covers the Workspace-specific setup and patterns. For general `snowflakeR` usage, see `vignette("getting-started")`.

How %%R Cells Work

Workspace Notebooks only support Python cells – there is no R kernel option. However, when you start a cell with `%%R`, you are using an IPython "cell magic" that tells the Python kernel: "don't execute this as Python – hand it to rpy2, which will execute it as R." The cell contents are pure R code; rpy2 passes it to the embedded R interpreter, runs it, and returns any output or plots back to the notebook. To you as the user, it feels like writing in an R notebook – the Python kernel orchestrating behind the scenes is an implementation detail you can ignore.

The setup cells (Steps 1-3 below) are the only Python code you need to run. After that, everything is `%%R`.

Architecture

In a Workspace Notebook, the data flow is:

```
Python Kernel (Jupyter)
  |-- rpy2 (Python-R bridge)
  |     |-- %%R magic cells (R code execution)
  |     |-- snowflakeR (R package, ML platform features)
  |     |     |-- reticulate (R-Python bridge)
  |     |           |-- snowflake-ml-python (Snowflake ML SDK)
  |     |                 |-- Snowpark Session (built-in)
  |     |
  |     |-- RSnowflake (R package, DBI/dbplyr, optional)
  |           |-- Snowflake SQL API v2 (reads, small writes)
  |           |-- reticulate -> write_pandas (bulk writes via SPCS path)
  |           |-- ADBC (optional, Arrow-native reads)
  |
  |-- Active Snowpark Session (auto-detected by sfr_connect() / RSnowflake)
```

`snowflakeR` uses `reticulate` to call the Snowflake ML Python SDK, while `RSnowflake` connects directly to Snowflake via HTTPS (no Python needed). Both work in Workspace Notebooks. For ML workflows (Model Registry, Feature Store), use `snowflakeR`. For standard DBI/dbplyr database operations, install `RSnowflake` and use `sfr_dbi_connection()`.

Setup

`snowflakeR` ships a **self-contained `notebooks/` directory** with all the scripts, config, and helpers needed. Upload the folder contents to your Workspace, then follow the steps below.

To find the directory after installing the package:

```r
system.file("notebooks", package = "snowflakeR")
```

Step 0: Configure (optional)

Workspace Notebooks do not auto-set database or schema. Each notebook has a `_config.yaml` where you can optionally override the session context. If omitted, `setup_notebook()` uses the Snowpark session's current database/schema/warehouse:

```yaml
# snowflaker_config.yaml -- all sections optional
context:
  warehouse: "MY_WAREHOUSE"
  database: "MY_DATABASE"
  schema: "MY_SCHEMA"
```

Table references use fully qualified names (`DATABASE.SCHEMA.TABLE`) via `sfr_fqn()`.

Step 1: Bootstrap the notebook (single cell)

Workspace Notebooks run in containers without R pre-installed. The `sfnb-multilang` toolkit handles installation. Each notebook directory includes an `sfnb_setup.py` bootstrap file. A single call to `setup_notebook()` handles everything:

```python
# Python cell -- run once per session (~45 sec fresh, ~2 sec cached)
from sfnb_setup import setup_notebook
setup_notebook(config="snowflaker_config.yaml", packages=["snowflakeR"])
```

This single call:

- Sets session context (from config or session defaults)
- Validates/creates the EAI with all required domains
- Installs R via `micromamba` + conda-forge and registers the `%%R` magic
- Installs R packages (from tarballs, URLs, or GitHub via `pak`)
- Exports SPCS OAuth env vars for RSnowflake DBI connectivity

Everything is installed in **user-space** – no `sudo` or root access is required.

Alternative (step-by-step): If you prefer separate steps:

```python
from sfnb_setup import install_r, install_r_packages
install_r(config="snowflaker_config.yaml")
install_r_packages(config="snowflaker_config.yaml", packages=["snowflakeR"])
```

Alternative (no sfnb_setup.py): Install the toolkit directly:

```python
!pip install sfnb-multilang
from sfnb_multilang import install
install(languages=["r"])
```

Licensing: All software installed by the toolkit is freely-licensed open-source. `micromamba` is BSD-3-Clause (developed by the mamba-org community, not affiliated with Anaconda, Inc.). All packages come from conda-forge, a community-driven open-source collection entirely separate from Anaconda's default repository. R is GPL-licensed, and CRAN packages carry their own open-source licenses. You can use all of this freely in commercial environments.

External Access Integration (EAI): The toolkit downloads packages from external hosts (`micro.mamba.pm`, `conda.anaconda.org`, `repo.anaconda.com`, and optionally `cloud.r-project.org`). Snowflake blocks outbound traffic by default, so you must attach an External Access Integration that allows these hosts. The toolkit can generate the required EAI SQL for you – see the network rules documentation.

Step 2: Install and load snowflakeR

```
%%R
options(repos = c(CRAN = "https://cloud.r-project.org"))

deps <- c("reticulate", "cli", "rlang")
for (pkg in deps) {
  if (!requireNamespace(pkg, quietly = TRUE))
    install.packages(pkg, type = "source", quiet = TRUE)
}

if (nzchar(Sys.getenv("SNOWFLAKER_PATH"))) {
  install.packages(Sys.getenv("SNOWFLAKER_PATH"), repos = NULL, type = "source")
} else {
  install.packages("pak", type = "source", quiet = TRUE)
  pak::pak("Snowflake-Labs/snowflakeR", ask = FALSE, upgrade = FALSE)
}
library(snowflakeR)
```

When running from the internal monorepo, `sfnb_setup.py` automatically sets `SNOWFLAKER_PATH` to the local package source. From the public repository, the `else` branch installs from GitHub via `pak`.

Installing additional R packages

You can also install packages interactively from `%%R` cells. Workspace Notebooks have no interactive stdin, so use this pattern to suppress prompts and compilation noise:

```
%%R
options(repos = c(CRAN = "https://cloud.r-project.org"))
install.packages("forecast", type = "source", quiet = TRUE)
suppressPackageStartupMessages(library(forecast))
```

- `options(repos = ...)` – suppresses the CRAN mirror selection prompt
- `type = "source"` – suppresses the "binary vs source" prompt (no Linux binaries on CRAN)
- `quiet = TRUE` – suppresses verbose compilation output
- `suppressPackageStartupMessages()` – suppresses startup banners

For packages that have their own interactive prompts (e.g., `pak::pak()`), use `ask = FALSE` or `upgrade = "never"`.

System library dependencies: R packages with compiled C/C++ code may depend on system libraries (e.g., `sf` needs `libgdal`). Since you don't have root access in the container, `install.packages()` will fail if those libraries are missing. Use conda-forge equivalents (e.g., `r-sf`) in your YAML config preset's `conda_packages` list instead – conda-forge bundles the required system libraries.

Persistence: Packages installed interactively don't persist across container recycles. Add regularly-used packages to a YAML config preset (see `configs/snowflaker.yaml` in the `sfnb-multilang` toolkit) for automatic reinstallation.

Step 5: Connect and set execution context

```
%%R
conn <- sfr_connect()   # auto-detects Workspace session

# Load config and set USE WAREHOUSE / DATABASE / SCHEMA
conn <- sfr_load_notebook_config(conn)
```

```r
# Use fully qualified names for all table operations
sfr_fqn(conn, "MY_TABLE")
#> "MY_DATABASE.MY_SCHEMA.MY_TABLE"
```

Connecting: Auto-detection

The biggest convenience in Workspace Notebooks: `sfr_connect()` automatically detects the active Snowpark session. No credentials, no configuration files.

```r
%%R
conn <- sfr_connect()
conn
#> <sfr_connection>
#>   environment: "workspace"
#>   auth: "session_token"
#>   account: "xy12345"
#>   database: "MY_DB"
```

Internally, `sfr_connect()` calls `get_active_session()` from the Snowpark Python library. If that succeeds (it always does in Workspace Notebooks), it wraps the session in an `sfr_connection` object.

When running locally, `sfr_connect()` falls back to `connections.toml` or explicit parameters. Your code works in both environments without changes.

%%R magic patterns

Basic R cell

Every cell starting with `%%R` has its contents executed as R code by rpy2. The R session persists across cells, so variables, loaded packages, and connections carry over.

```r
%%R
library(dplyr)
result <- sfr_query(conn, "SELECT * FROM my_table LIMIT 10")
head(result)
```

Passing data from Python to R (-i flag)

Since `%%R` cells run inside a Python kernel, you can pass variables between Python and R using the `-i` (input) and `-o` (output) flags. rpy2 handles type conversion automatically (pandas DataFrames become R data.frames, scalars map to R vectors, etc.).

```r
# Python cell
import pandas as pd
py_data = pd.DataFrame({"x": [1, 2, 3], "y": [4, 5, 6]})
```

```r
%%R -i py_data
# py_data is now an R data.frame
str(py_data)
summary(py_data)
```

Passing data from R to Python (-o flag)

```r
%%R -o r_result
r_result <- sfr_query(conn, "SELECT * FROM my_table")
```

```
# Python cell -- r_result is now a pandas DataFrame
r_result.head()
```

Plot sizing (-w and -h flags)

Workspace Notebooks need explicit plot dimensions:

```
%%R -w 700 -h 450
library(ggplot2)

p <- ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point() +
  theme_minimal()

print(p)  # print() is required to render in Workspace
```

Output helpers

Workspace Notebooks add extra line breaks to R output. `snowflakeR` includes helper functions for cleaner display:

Function	Replaces	Description
`rprint(x)`	`print(x)`	Clean printing of any object
`rview(df, n)`	`head(df)`	View data frame with optional row limit
`rglimpse(df)`	`glimpse(df)`	Compact data frame structure
`rcat(...)`	`cat(...)`	Clean `cat()` output

```
%%R
df <- sfr_query(conn, "SELECT * FROM my_table")

# Instead of print(df):
rprint(df)

# Instead of head(df, 10):
rview(df, n = 10)

# Instead of str(df):
rglimpse(df)
```

Bulk Write Performance

When writing large data from R in Workspace Notebooks, RSnowflake automatically routes through ADBC (if installed) or the Python Snowpark session as a fallback. Both paths use the **internal SPCS gateway** (`SNOWFLAKE_HOST`) with SPCS OAuth authentication – zero-config, no public endpoint or PAT required.

The preferred data flow for bulk writes in Workspace is:

```
R data.frame  -->  ADBC bulk ingest (PUT + COPY INTO)
                   via internal SPCS gateway (SNOWFLAKE_HOST)
                   Arrow-native, ~7s for 50K rows
```

Fallback (when ADBC is not installed):

```
R data.frame  -->  reticulate::r_to_py()  -->  pandas DataFrame
```

```
                    |
        session.write_pandas()
                    |
Internal SPCS gateway (PUT + COPY INTO)
~2.5s for 50K rows
```

Outside Workspace, RSnowflake uses ADBC (if available) for bulk writes via the public endpoint. See the RSnowflake `getting-started` vignette for the full `upload_method` options table.

PAT management

Most `snowflakeR` operations use the Snowpark session directly (no extra auth needed). A **Programmatic Access Token (PAT)** is only required for:

- `sfr_predict_rest()` – pure-R REST inference against SPCS endpoints (the Workspace session token is not authorised for SPCS ingress)

For REST inference in local environments, create a PAT in **Snowsight > Account > Programmatic Access Tokens** and set:

```
Sys.setenv(SNOWFLAKE_PAT = "your_token")
```

For Workspace Notebooks, use `sfr_predict()` (Python-bridge predict) instead of `sfr_predict_rest()` – it routes through the Snowpark session and requires no separate token.

Local vs Workspace differences

Aspect	Local (RStudio/Posit)	Workspace Notebook
R kernel	Native R	Python + `%%R` magic
Authentication	`connections.toml`, key-pair, browser	Auto-detected session
Package install	`install.packages()`	`micromamba` + `install.packages()`
Plotting	Native plot pane	`%%R -w W -h H` + `print(p)`
Output	Standard	Use `rprint()` / `rview()` helpers
Session persistence	Persistent	Re-setup each session (~1-2 min)
Python interop	`reticulate`	`rpy2` (bidirectional)
`sfr_connect()`	Reads credentials	Wraps active session

The key design goal: **your `snowflakeR` code is identical** in both environments. Only the setup differs.

Typical workflow

```python
# Cell 1 (Python): Bootstrap R, packages, EAI, and context
from sfnb_setup import setup_notebook
setup_notebook(config="snowflaker_config.yaml", packages=["snowflakeR"])
```

```r
%%R
# Cell 2 (R): Load and connect
library(snowflakeR)
conn <- sfr_connect()

fs   <- sfr_feature_store(conn)
reg  <- sfr_model_registry(conn)
```

```
%%R
# Cell 3 (R): Generate training data
training <- sfr_generate_training_data(
  fs,
  spine = "SELECT customer_id, churned FROM labels",
  features = list(list(name = "CUST_FEATURES", version = "v1")),
  spine_label_cols = "churned"
)
```

```
%%R -o predictions
# Cell 4 (R): Train and predict
model <- glm(churned ~ ., data = training, family = binomial)
predictions <- sfr_predict_local(model, training)
```

```
# Cell 5 (Python): Visualise or further processing
predictions.head()
```

Interactive notebooks

For runnable notebook versions of the examples above, see the self-contained `notebooks/` directory included with the package:

```
system.file("notebooks", package = "snowflakeR")
```

Notebook	Environment	Description
`workspace_quickstart.ipynb`	Workspace	Connection, config, queries, ggplot2
`local_quickstart.ipynb`	Local (RStudio, Jupyter)	Same content, native R kernel
`workspace_model_registry.ipynb`	Workspace	Model Registry: log, deploy, serve R models
`workspace_model_consumption.ipynb`	Workspace	Consume Python models from R (`sfr_predict_sql`)
`workspace_model_monitoring.ipynb`	Workspace	Model monitoring: drift, performance, segments
`workspace_feature_store.ipynb`	Workspace	Feature Store: entities, views, training data
`workspace_forecasting_demo.ipynb`	Workspace	Time series forecasting with forecast + ggplot2
`workspace_credit_risk_setup.ipynb`	Workspace	Credit risk demo: data preparation and setup
`workspace_credit_risk_demo.ipynb`	Workspace	Credit risk demo: training and registry
`workspace_parallel_spcs_setup.ipynb`	Workspace	Parallel SPCS lab: synthetic data and setup
`workspace_parallel_spcs_demo.ipynb`	Workspace	Parallel SPCS lab: driver (tasks / queue patterns)
`workspace_parallel_spcs_monitoring.ipynb`	Workspace	Parallel SPCS lab: SQL monitoring in a second tab
`workspace_dosnowflake.ipynb`	Workspace	doSnowflake: `foreach` backends and infrastructure
`workspace_marketing_setup.ipynb`	Workspace	Marketing demo: setup
`workspace_marketing_demo.ipynb`	Workspace	Marketing demo: walkthrough
`local_model_registry.ipynb`	Local	Model Registry for local R environments

Notebook	Environment	Description
`local_feature_store.ipynb`	Local	Feature Store for local R environments
`local_forecasting_demo.ipynb`	Local	Forecasting demo for local R environments

Supporting files in the same directory include per-notebook `snowflaker_*.yaml` configs (for example `snowflaker_parallel_spcs_config.yaml` for the parallel lab), `sfnb_setup.py`, `PARALLEL_SPCS_DEMO.md`, `parallel_lab_config.py`, and optional `streamlit_parallel_demo_monitor.py`. See `README.md` in that folder for the authoritative list.

Workspace notebooks use `setup_notebook()` for context and EAI, and `sfr_fqn()` for fully qualified table names. Upload the entire folder to your Workspace or copy to a local directory to get started.

Parallel and distributed R in Workspace

SPCS-backed parallelism uses compute pools, image repositories, and (for queue mode) Hybrid Tables configured in your account. The shipped lab wires these through **`snowflaker_parallel_spcs_config.yaml`** and shared Python helpers (`parallel_lab_config.py`).

Suggested order:

1. **`workspace_parallel_spcs_setup.ipynb`** – creates or validates lab objects and session context.
2. **`workspace_parallel_spcs_demo.ipynb`** – driver notebook: run `registerDoSnowflake()` in `tasks` or `queue` mode (adapt paths and pool names to your environment).
3. **`workspace_parallel_spcs_monitor.ipynb`** (optional) – open in a **second tab** to run SQL-only monitoring while the demo notebook runs.

Read **`PARALLEL_SPCS_DEMO.md`** in the notebooks folder for the monitoring pattern and how bundled Model Registry inference fits the lab. Optionally run **`streamlit_parallel_demo_monitor.py`** alongside the driver; it reads the same YAML as the notebooks.

For a shorter walkthrough focused on **`foreach`** and setup helpers only, use **`workspace_dosnowflake.ipynb`**.

Conceptual overview and function-level references: `vignette("parallel-dosnowflake")`.

Troubleshooting

Issue	Solution
`rpy2` not found	Run `from sfnb_setup import setup_notebook; setup_notebook()`
`%%R` not recognised	Run `%load_ext rpy2.ipython`
Extra line breaks in output	Use `rprint()`, `rview()`, `rglimpse()`
Package not found	Add to a YAML config preset or install interactively
`sfr_connect()` fails	Ensure the Python kernel has an active Snowpark session
Plot not showing	Add `print(p)` and use `-w` / `-h` flags